

VOLTSTREAM (Week 1)

Intelligent Energy Management Platform

PreboardingTasks(Python/Git/HTML/CSS/JS/Docker/FastAPI/React/ReactRouter) + Travel Journal (Major Project) + Deployment (Cloud Run + FireBase)

VoltStream SourceCode:

<https://github.com/MYethishwar/AIengineering/tree/main/Week%201%20Pre-boarding%20projects>

Travel Journal SourceCode:

<https://github.com/MYethishwar/AI-engineering/tree/main/Travel%20Journal%20Application>

VoltStream Project Overview:

VoltStream is an energy monitoring and management application created to track device usage, monitor analytics, view billing information, and manage connected devices from a single dashboard. The project combines frontend and backend technologies to create a complete full-stack application with cloud deployment support.

The application is designed in a way where users can monitor smart devices, analyze usage trends, and understand energy consumption patterns through visual dashboards and reports.

Travel Journal Project Overview:

The Travel Journal Application is a full-stack web application developed to help users document, organize, and manage their travel experiences digitally. The application allows users to create travel journals, upload trip photos, track expenses, maintain wishlists for future destinations, and store important travel memories in a structured manner.

The project is developed using a modern technology stack consisting of React.js for the frontend, FastAPI for the backend, and MongoDB as the database. The application follows a REST API architecture and supports CRUD operations for managing travel data efficiently.

Use Case

The project can be used in homes, offices, or small businesses where multiple devices or sensors are connected to utility systems. Instead of checking different systems separately, users can monitor everything from one interface.

Users can:

- View all connected devices
- Track current usage and activity
- Analyse consumption patterns using charts
- View billing-related information
- Add or manage devices
- Understand technical terms and metrics used inside the application

The main purpose of the project is to simplify energy monitoring and make usage data easier to understand.

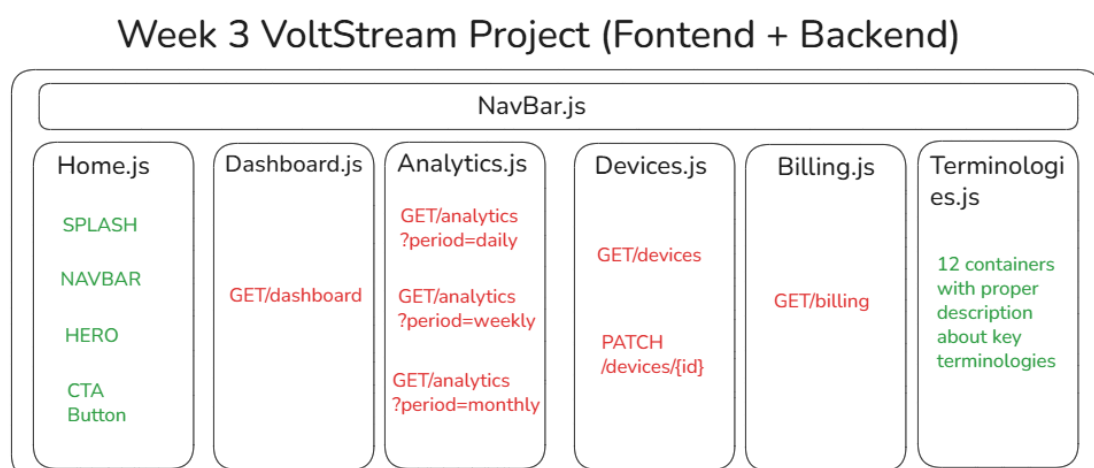
Project Architecture

The project follows a frontend-backend architecture.

The frontend handles everything users interact with in the browser such as pages, navigation, charts, and dashboards.

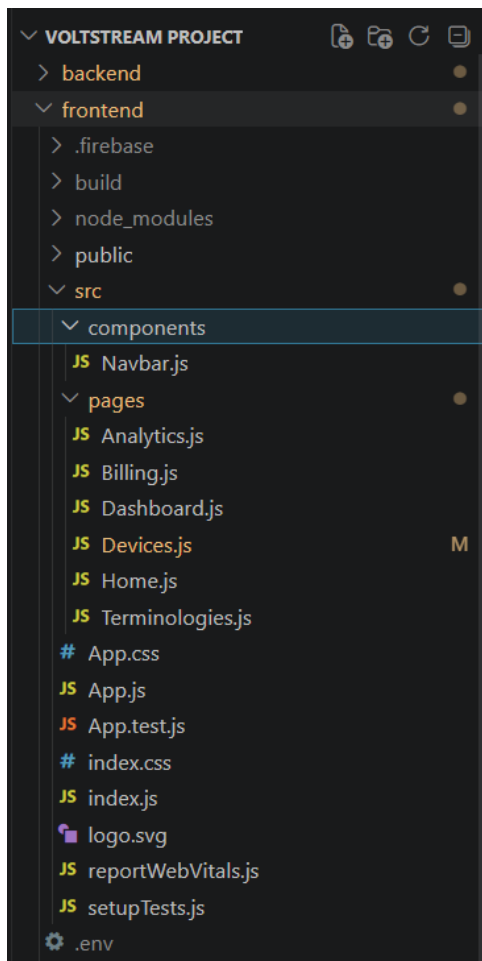
The backend handles API requests, business logic, and data processing. It sends data to the frontend whenever users interact with the application.

The overall structure looks like this:



The frontend and backend work independently but communicate through API requests.

Frontend Structure



The frontend is developed using React and Tailwind CSS.

It contains different pages and reusable components that build the user interface.

Main Frontend Files

public/index.html

This is the base HTML file loaded in the browser.

src/index.js

This file starts the React application and connects React with the HTML root element.

src/App.js

This acts as the main application container. It handles routing and page rendering.

src/components/Navbar.js

This contains the navigation bar used across different pages.

Frontend Pages

Home Page

Acts as the landing page or welcome screen.

Dashboard Page

Displays overall device information, quick statistics, and summaries.

Analytics Page

Shows charts, trends, and visual analysis of energy usage.

Billing Page

Displays billing details, usage summaries, and cost-related information.

Devices Page

Allows users to manage connected devices and view device details.

Terminologies Page

Contains explanations of technical terms and application-related definitions.

Styling & Configuration

Tailwind CSS

Used for responsive styling and UI design.

package.json

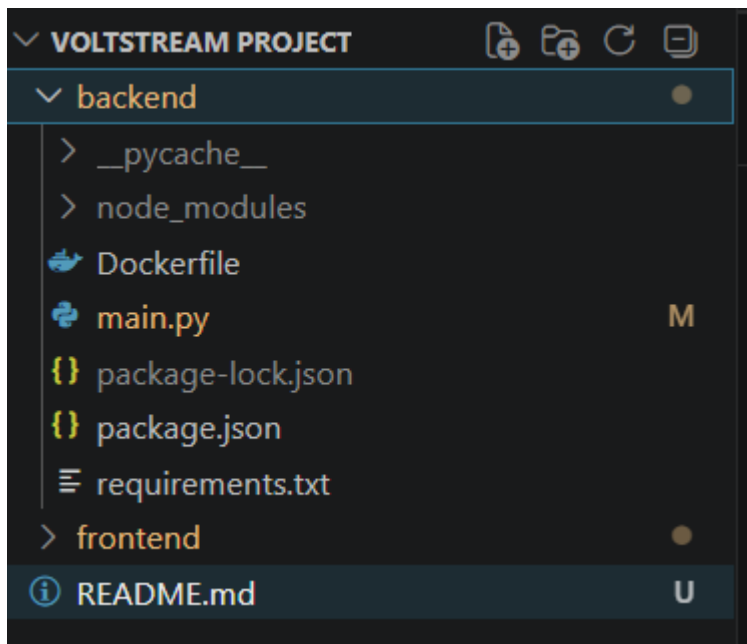
Contains frontend dependencies and scripts such as:

- npm start
- npm run build

firebase.json

Used for Firebase Hosting deployment configuration.

Backend Structure



The backend is developed using Python.

It handles:

- API creation
- Request processing
- Data communication
- Backend logic

Main Backend Files

main.py

This is the core backend file where APIs and request handling logic are written.

Example functionalities include:

- Fetching device data
- Returning analytics information
- Sending billing details
- Handling user requests

requirements.txt

Contains Python dependencies required for the backend.

Dockerfile

Used to containerize the backend application for deployment.

Data Workflow

The frontend and backend communicate through APIs.

The workflow is:

1. User opens the frontend application.
2. React renders the required page.
3. Frontend sends API requests to the backend.
4. Backend processes the request.
5. Backend returns data in JSON format.
6. Frontend receives the response.
7. UI updates dynamically with the received data.

This workflow happens continuously whenever users interact with dashboards, analytics, billing, or devices.

Local Development Workflow

Running Backend Locally

- `cd backend`
- `pip install -r requirements.txt`
- `python main.py`

The backend starts running on a local server.

Running Frontend Locally

- `cd frontend`
- `npm install`
- `npm start`

The frontend starts running in the browser using the React development server.

Deployment Workflow

Frontend Deployment

The frontend is deployed using Firebase Hosting.

Steps followed:

1. Create production build using:

```
npm run build
```

2. Deploy using:

```
firebase deploy --only hosting
```

Backend Deployment

The backend is deployed using Google Cloud Run.

Steps followed:

1. Build Docker image
2. Push image to Google Cloud
3. Deploy service using Cloud Run

Example commands:

```
gcloud builds submit --tag gcr.io/PROJECT_ID/voltstream-backend
```

```
gcloud run deploy
```

Technologies Used

Frontend

- React
- Tailwind CSS
- Firebase Hosting
- Node.js
- npm

Backend

- Python
 - Docker
 - Google Cloud Run
-

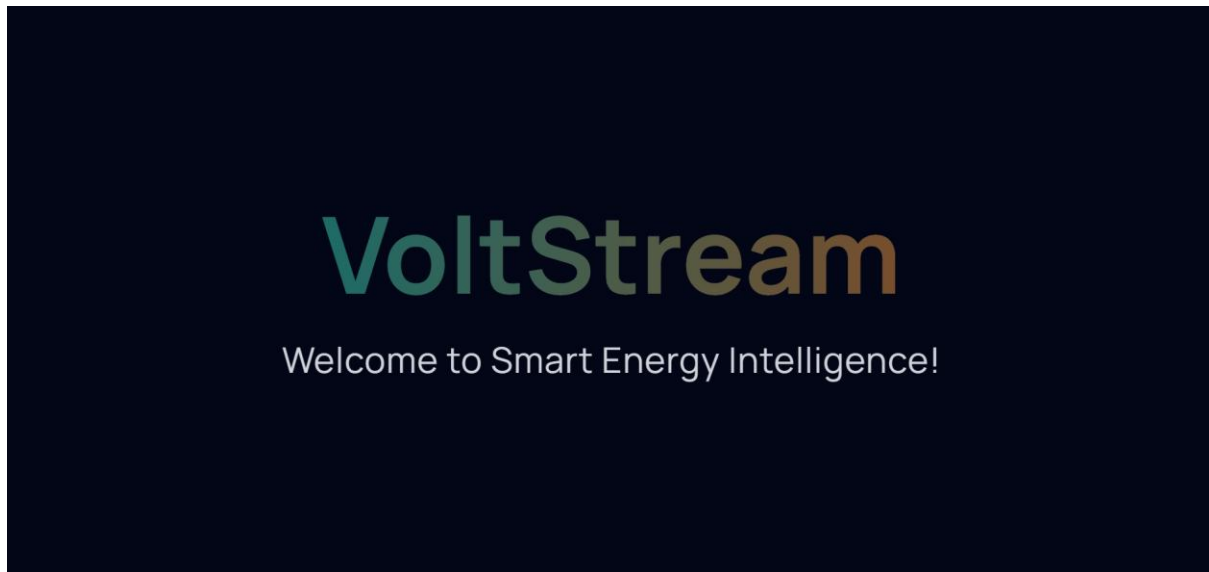
Learning & Understanding

This project helped in understanding:

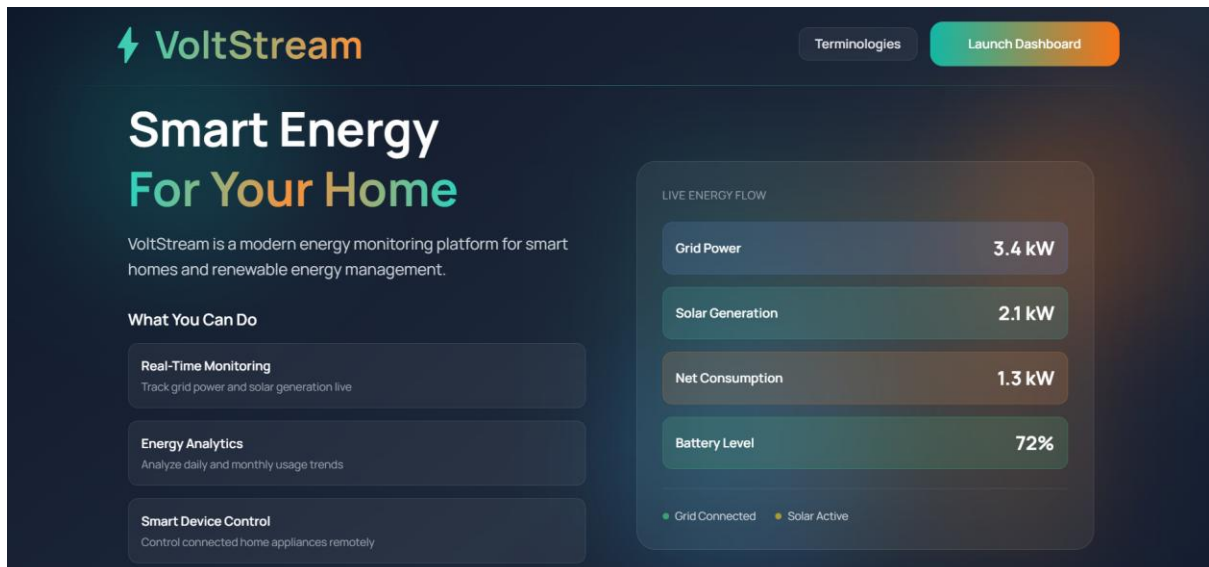
- React project structure
 - Component-based frontend development
 - Routing and navigation
 - Tailwind CSS styling
 - Backend API creation
 - Frontend-backend communication
 - Docker basics
 - Firebase deployment
 - Google Cloud deployment
 - Production build workflows
-

Frontend Output:

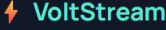
Opening Splash Page:



Home Page:



Terminologies Page:

 [Dashboard](#) [Analytics](#) [Devices](#) [Billing](#)

VoltStream Terminologies

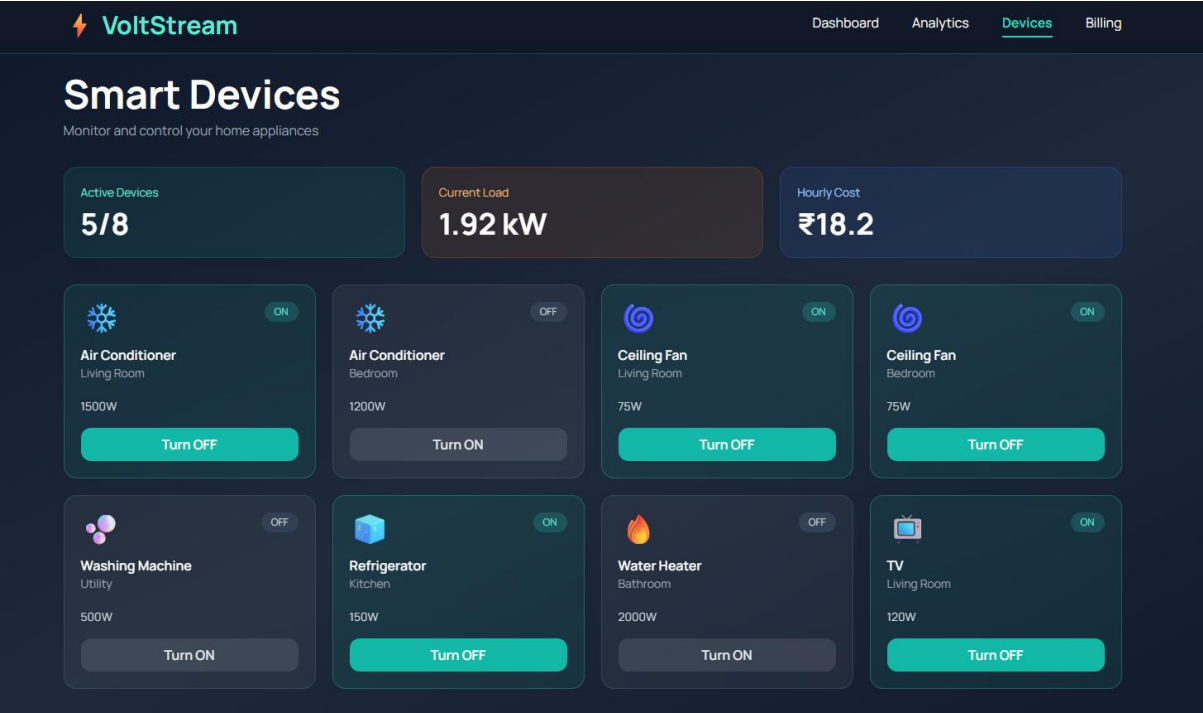
Key concepts used throughout the VoltStream energy platform

Grid Power Electricity supplied from the utility grid to power the home.	Solar Generation Energy produced through rooftop solar panels in real time.	Net Consumption Final electricity usage after subtracting solar generation.
Battery Level Current percentage of stored backup energy available.	Energy Flow Movement of electricity between the grid, solar system, and home.	Solar Coverage Percentage of total electricity demand fulfilled by solar energy.
CO₂ Saved Estimated carbon emissions reduced through renewable energy usage.	Current Load Total active power currently consumed by running devices.	Smart Devices Connected appliances that can be remotely monitored and controlled.
Active Devices Number of appliances currently powered ON in the system.	Hourly Cost Estimated electricity expense generated every hour.	Energy Analytics Historical analysis of energy usage and solar generation patterns.

Analytics Page:



Devices Page:



Backend Output:

VoltStream API 1.0.0 OAS 3.1

/openapi.json

default ^		
GET	/ Root	▼
GET	/dashboard Get Dashboard	▼
GET	/analytics Get Analytics	▼
GET	/devices Get Devices	▼
PATCH	/devices/{device_id} Update Device	▼
GET	/billing Get Billing	▼

VoltStream Deployment Steps

Backend Deployment using Docker + Google Cloud Run

Step 1 – Create Backend Files

First, the backend required:

- `main.py`
- `requirements.txt`
- `Dockerfile`

The `requirements.txt` file contains Python dependencies.

The `Dockerfile` is used to create a Docker image for deployment.

Step 2 – Login to Google Cloud

- `gcloud auth login`

This opens a browser and connects the local system with the Google Cloud account.

Step 3 – Select Google Cloud Project

- `gcloud config set project voltstream-yethishwar`

OR

- `gcloud config set project quixotic-dynamo-495704-k4`

This sets the active Google Cloud project.

Step 4 – Enable Required Services

These services are needed for deployment.

- `gcloud services enable run.googleapis.com`
- `gcloud services enable artifactregistry.googleapis.com`
- `gcloud services enable cloudbuild.googleapis.com`

Step 5 – Build Docker Image

Move inside backend folder:

➤ `cd backend`

Build Docker image:

➤ `docker build -t voltstream-yethishwar-backend .`

This creates the backend Docker image locally.

You successfully built images like:

- `voltstream-yethishwar-backend`
- `voltstream-backend`

Step 6 – Create Artifact Registry Repository

Create repository:

➤ `gcloud artifacts repositories create my-repo \`
`--repository-format=docker \`
`--location=us-central1`

OR

➤ `gcloud artifacts repositories create voltstream-repo \`
`--repository-format=docker \`
`--location=asia-south1`

Artifact Registry stores Docker images in Google Cloud.

Step 7 – Configure Docker Authentication

Initially you got this error:

Unauthenticated request.
`artifactregistry.repositories.uploadArtifacts`

This happened because Docker authentication was not configured properly.

You fixed it using:

- `gcloud auth configure-docker asia-south1-docker.pkg.dev`

OR

- `gcloud auth configure-docker us-central1-docker.pkg.dev`

This connected Docker with Google Cloud authentication.

After this step, Docker push started working correctly.

Step 8 – Tag Docker Image

You tagged the local Docker image with Artifact Registry path.

Example:

- `docker tag voltstream-yethishwar-backend us-central1-docker.pkg.dev/voltstream-yethishwar/my-repo/voltstream-yethishwar-backend`

OR

- `docker tag voltstream-backend asia-south1-docker.pkg.dev/quixotic-dynamo-495704-k4/voltstream-repo/voltstream-backend`

This prepares the image for cloud upload.

Step 9 – Push Docker Image to Artifact Registry

Push image:

- `docker push us-central1-docker.pkg.dev/voltstream-yethishwar/my-repo/voltstream-yethishwar-backend`

OR

- `docker push asia-south1-docker.pkg.dev/quixotic-dynamo-495704-k4/voltstream-repo/voltstream-backend`

You successfully pushed Docker images to Artifact Registry.

Output showed:

- Pushed
 - latest: digest
 - image uploaded successfully
-

Step 10 – Deploy Backend to Cloud Run

Deploy command:

- `gcloud run deploy voltstream-yethishwar-backend \`
`--image us-central1-docker.pkg.dev/voltstream-yethishwar/my-`
`repo/voltstream-yethishwar-backend \`
`--platform managed \`
`--region us-central1 \`
`--allow-unauthenticated`

OR

- `gcloud run deploy voltstream-backend \`
`--image asia-south1-docker.pkg.dev/quixotic-dynamo-495704-`
`k4/voltstream-repo/voltstream-backend \`
`--platform managed \`
`--region asia-south1 \`
`--allow-unauthenticated`

Cloud Run successfully deployed the backend.

Generated service URL:

<https://voltstream-backend-235566209668.asia-south1.run.app>

and

<https://voltstream-yethishwar-backend-536159688445.us-central1.run.app>

These URLs became the live backend APIs.

Local Backend Testing

You also tested backend locally using Uvicorn.

Command used:

➤ `python -m uvicorn main:app --reload`

Backend started locally on:

<http://127.0.0.1:8000>

The logs showed:

- Application startup complete
- GET requests working
- Auto reload working after file changes

This confirmed backend APIs were functioning correctly before deployment.

Frontend Deployment using React + Firebase

Step 1 – Run Frontend Locally

Move to frontend folder:

➤ `cd frontend`

Run frontend:

➤ `npm start`

Frontend successfully started at:

<http://localhost:3000>

This was used to test and fix frontend issues before deployment.

Step 2 – Install Firebase CLI

Install Firebase tools globally:

➤ `npm install -g firebase-tools`

This installs Firebase deployment commands.

Step 3 – Login to Firebase

➤ `firebase login`

This connects Firebase CLI with Google account.

Step 4 – Initialize Firebase

➤ `firebase init`

During setup:

- Hosting option selected
 - Build folder selected
 - React SPA configuration enabled
-

Step 5 – Configure Build Directory

While running Firebase initialization:

- Set public directory as:

➤ `build`

This tells Firebase to deploy React production files.

Step 6 – Create Production Build

Run:

➤ `npm run build`

This creates optimized production files inside:

frontend/build

The build folder contains:

- Optimized HTML
 - CSS
 - JavaScript bundles
-

Step 7 – Deploy Frontend

Deploy command:

➤ `firebase deploy`

OR

`firebase deploy --only hosting`

Firebase uploads the React build files and provides a live frontend URL.

Complete Workflow Followed

The overall deployment workflow became:

1. Create backend APIs
2. Create Dockerfile and requirements
3. Build Docker image
4. Configure Google Cloud
5. Push image to Artifact Registry

6. Deploy backend using Cloud Run
 7. Test backend locally and on cloud
 8. Build React frontend
 9. Configure Firebase Hosting
 10. Generate production build
 11. Deploy frontend using Firebase
 12. Connect frontend with deployed backend API
-

Hosting URL's

Backend: <https://voltstream-yethishwar-backend-536159688445.us-central1.run.app>

Frontend: <https://yethishwar-voltstream.web.app/>

Travel Journal Application

Project Overview

The Travel Journal Application is a full-stack web application developed to help users document, organize, and manage their travel experiences digitally. The application allows users to create travel journals, upload trip photos, track expenses, maintain wishlists for future destinations, and store important travel memories in a structured manner.

The project is developed using a modern technology stack consisting of React.js for the frontend, FastAPI for the backend, and MongoDB as the database. The application follows a REST API architecture and supports CRUD operations for managing travel data efficiently.

Features

- Create and manage travel journal entries
- Store trip details such as location, dates, and descriptions
- Track travel expenses and calculate total expenditure
- Upload and manage travel photos
- Add favorite moments and visited places
- Maintain a wishlist for future travel destinations
- Search journals by location or title
- Responsive frontend user interface

- Backend API integration with MongoDB

Frontend

The frontend is built using React.js and Tailwind CSS. It provides an interactive and responsive user interface for users to interact with the application. React Router is used for navigation between pages, while Axios or Fetch API is used to communicate with backend APIs.

Backend

The backend is developed using FastAPI, which provides high-performance asynchronous API handling. The backend manages journal entries, wishlist data, image uploads, and database communication.

Database

MongoDB is used as the primary database for storing:

- Journal entries
- Expense details
- Wishlist items
- Uploaded image references

Motor AsyncIO driver is used for asynchronous communication between FastAPI and MongoDB.

Image Upload System

The application supports image uploads using FastAPI file handling features. Uploaded images are stored in a local uploads directory and served as static files.

Docker Support

The project is containerized using Docker. A multi-stage Dockerfile is used to:

1. Build the React frontend
2. Install Python backend dependencies
3. Combine frontend and backend into a single container

Deployment

The application can be deployed using:

- Docker Hub
- Google Cloud Run
- Firebase Hosting (frontend)
- Local Docker containers

Conclusion

The Travel Journal Application demonstrates the implementation of a modern full-stack architecture using React, FastAPI, MongoDB, and Docker. The project provides practical experience in frontend